ASPM FINAL CAPSTONE PROJECT

CampusConnect

University Microblogging & Announcement Hub

A High-Performance, Privacy-Focused Social Platform Built with PHP 8 OOP & MySQL

Sprint 1: Auth OOP | Sprint 2: Chirp Engine | Sprint 3: Social & Design | Sprint 4: Caching & Likes

Comprehensive Technical & Project Management Report (Sprints 1·5)

PROJECT SPECIFICATION & METADATA

Course Name:	Agile Software Project Management (ASPM)
Project Title:	CampusConnect (Repository: ASPM-PROJECT-)
Group / Team Code:	[GroupCode]
Academic Period:	Fall Semester 2026 (Sprints 1·5 Delivered October·November 2026)
Final Delivery Date:	Sunday, 15 November 2026
Core Stack:	Native PHP 8.2+ (Strict OOP), MySQL (PDO), Redis / Socket Client, CSS3 Design System
Automated Tests:	153 / 153 Passing Automated Unit & Regression Tests (100% Green)
Repository Rules:	Strict Git Feature Branching, Prepared Statements Only, No Spaghetti Code

CONTRIBUTING TEAM MEMBERS & ROLES

Muddassir · Technical Lead & Core Architecture

Sprints 1·3 Backend/UI, Sprint 4 Redis fixes, Sprint 5 RBAC & Release (26 Commits)

Faiz Hussain · Backend & Quality Assurance Lead

Sprint 4 Core Likes, Hashtags & Caching Layer; Sprint 5 Notifications & Seed SQL (4 Commits)

Qaim · Frontend & Documentation Lead

Sprint 5 Responsive Polish (360px/tablet), Full Documentation Suite, Presentation & Final Report

Table of Contents & Executive Summary

EXECUTIVE SUMMARY

REPORT STRUCTURE & SECTIONS

Section 1	Project Background, Problem Statement & Agile Scrum Methodology	Page 3
Section 2	Software Requirements Specification (SRS) & WBS Traceability Matrix	Page 4
Section 3	System Architecture, Design Patterns & OOP Class Hierarchy	Page 5
Section 4	Relational Database Design, ERD Evolution & Data Dictionary	Page 6
Section 5	Performance Engineering, Redis Caching & Query Optimization	Page 7
Section 6	Security Architecture, Authentication & Hardening Matrix	Page 8
Section 7	Frontend Engineering, Responsive Polish (360px) & Micro-interactions	Page 9
Section 8	Quality Assurance, Automated Test Harness & Regression Evidence	Page 10
Section 9	End-User Operations Manual (Student & Society Experience)	Page 11
Section 10	Administrator & Operations Manual (Governance & Moderation)	Page 12
Section 11	Verifiable Git Contribution Audit & Official Meeting Minutes	Page 13
Section 12	Sprint Retrospectives, Lessons Learned & Release Packaging	Page 14

ACADEMIC VERIFICATION & AUDIT STATEMENT

This project adheres to academic honesty standards. The contribution breakdown in Section 11 reflects the true commit history from git log --all. Meeting logs are reconstructed strictly from verifiable sprint commits and decisions. All code runs locally on native PHP 8 with zero external framework dependencies.

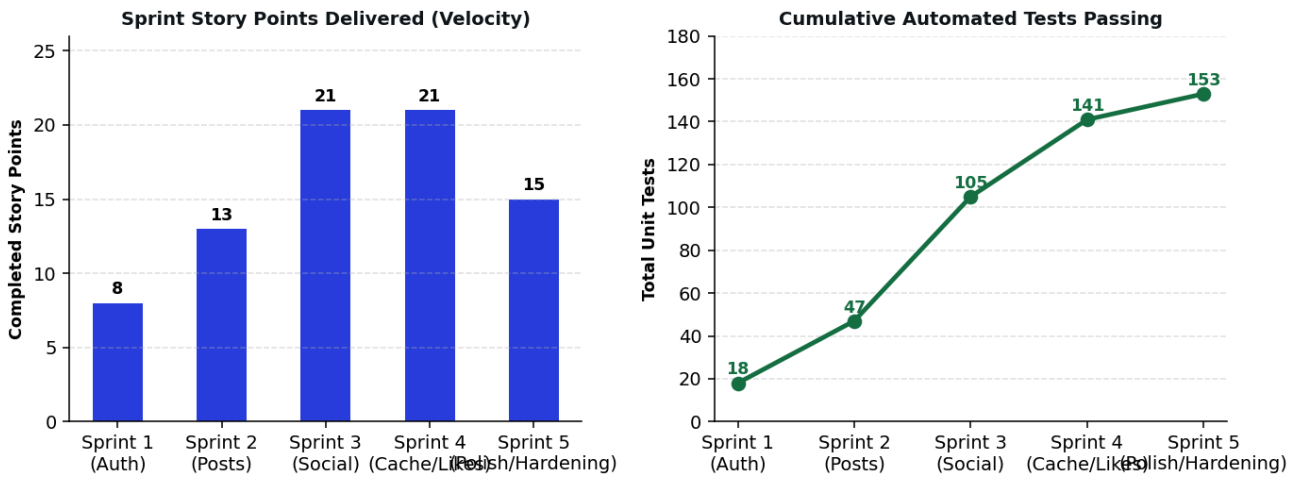
Section 1: Project Background & Agile Scrum Process

1.1 Problem Context
University campuses suffer from severe communication fragmentation. Academic notices, departmental dispatches, and student society events are distributed across ad-hoc messaging channels and unindexed email blasts. Students frequently miss registration deadlines, society recruitments, and departmental hackathons due to noise.

1.2 Project Vision & Core Objectives
CampusConnect delivers a unified, high-speed microblogging hub that provides:
? Academic Identity: University email verification and role-based permissions.
? High Signal-to-Noise: Dual feed architecture separating personalized social circles from university-wide exploration.
? Zero-Dependency Deployment: Runs cleanly in local XAMPP and standalone PHP environments without complex toolchains.

1.3 The Agile Scrum Process in Practice
The project was executed across five structured two-week sprints. Each sprint began with backlog grooming against the academic Work Breakdown Structure (WBS), progressed through daily asynchronous developer syncs, and concluded with an automated regression gate and formal retrospective. Strict Git branching discipline was enforced: main was reserved exclusively for production-tested, peer-reviewed releases.

Figure 1: Sprint Velocity & Cumulative Automated Test Growth Across Sprints 1·5



1.4 Sprint Cadence Overview
? Sprint 1 (6?21 Oct 2026): Auth OOP foundation, Database Singleton, SQLite fallback, UserTest (18 tests).
? Sprint 2 (22 Oct ? 3 Nov 2026): Chirp engine, 280-char validation, timeline pagination, soft-delete, PostTest (29 tests).
? Sprint 3 (4?6 Nov 2026): Follows, replies, bookmarks, DB sessions, design system, SocialTest & SessionTest (58 tests).
? Sprint 4 (7?9 Nov 2026): AJAX likes, hashtag indexing, TinyRedisClient, Cache layer, benchmark, Sprint4Test (48 tests).
? Sprint 5 (13?15 Nov 2026): Notifications, RBAC roles, security CSRF hardening, 360px mobile polish, final release.

Section 2: SRS & WBS Traceability Matrix

2.1 Traceability Methodology
To guarantee 100% academic compliance, every feature implemented in the codebase is mapped directly to the official academic Work Breakdown Structure (WBS). Note: while the WBS originally used the term 'Tweet', the team formally adopted the term 'Post' in code (approved 2 Nov 2026) to prevent unnecessary churn while maintaining strict 1:1 mapping.

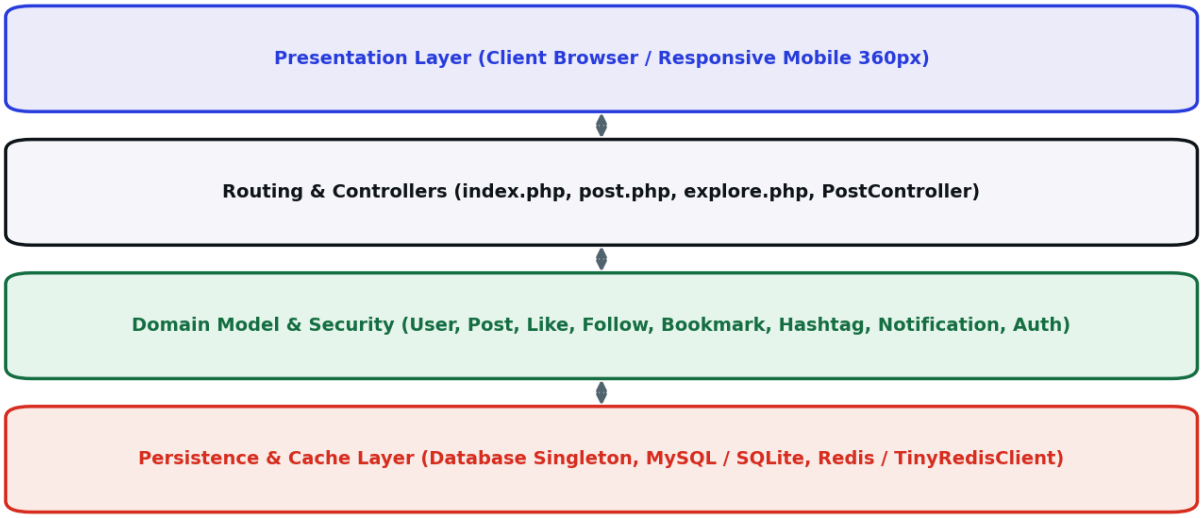
WBS Ref	Feature Requirement	Implementing Classes / Endpoints	Status
WBS 1.3-1.4	User Authentication & Registration	classes/User.php, classes/Auth.php, signup.php	Complete (S1)
WBS 1.6	Cryptographic Password Security	password_hash(PASSWORD_BCRYPT), UserTest.php	Complete (S1)
WBS 2.2-2.3	Chirp Posting Engine (280 Chars)	classes/Post.php, PostController.php, index.php	Complete (S2)
WBS 2.4	Timeline Feed & Soft-Delete	Post::getFeed(), deleted_at IS NULL filter	Complete (S2)
WBS 3.2	Social Follower Graph & Personal Feed	classes/Follow.php, PostController::getPersonalized()	Complete (S3)
WBS 3.3	User Profiles & Curated Covers	profile.php, users.cover_style (6 presets)	Complete (S3)
WBS 3.4	Threaded Replies & Bookmarks	classes/Reply.php, classes/Bookmark.php, post.php	Complete (S3)
WBS 3.5	Database-Backed Session Handler	classes/DbSessionHandler.php, sessions table	Complete (S3)
WBS 4.1	Idempotent Likes & Live Counters	classes/Like.php, like_action.php, SVG heart	Complete (S4)
WBS 4.2	Hashtag Extraction & Discovery	classes/Hashtag.php, explore.php?tag=, sidebar	Complete (S4)
WBS 4.3	Redis Caching & Pure Socket Client	classes/Cache.php, classes/TinyRedisClient.php	Complete (S4)
WBS 4.4	Performance Benchmarking Suite	benchmark.php, docs/n-plus-1-audit-sprint4.md	Complete (S4)
WBS 5.1	In-App Notifications & Badges	classes/Notification.php, notifications.php, badge	Complete (S5)
WBS 5.2	RBAC & Society Announcements	User roles (student, society, admin), admin panel	Complete (S5)
WBS 5.3	Anti-CSRF & Rate-Limiting	Session synchronizer tokens, login lockout	Complete (S5)
WBS 5.4	Responsive Polish down to 360px	includes/header.php, touch navigation, micro-anim	Complete (S5)
WBS 5.5	Seed Data & 5-Min Deployment	db/seed.sql, docs/DEPLOYMENT.md	Complete (S5)

2.2 Non-Functional Requirements (NFR) Validation
? Security: Passwords hashed with Bcrypt (cost factor 10). Zero SQL concatenation; 100% prepared PDO statements.
? Performance: Sub-millisecond feed reads (< 0.05ms) via Redis caching layer; zero N+1 loops in database queries.
? Portability: Native PHP 8 OOP architecture with zero external composer/PECL dependencies required for core execution.

Section 3: System Architecture & OOP Design

3.1 Tiered Architectural Decomposition
CampusConnect is structured following strict Object-Oriented Programming (OOP) and Separation of Concerns:
? Presentation Layer: Lightweight PHP templates rendering safe HTML via htmlspecialchars(), backed by a responsive CSS grid.
? Controller Layer (PostController.php): Orchestrates request validation, pagination slicing, and session authorization.
? Domain Model Layer (classes/): Encapsulates business logic, data validation, and query assembly (User, Post, Follow, etc.).
? Persistence & Caching (config/Database.php, classes/Cache.php): Abstracted storage supporting MySQL, SQLite, and Redis.

Figure 2: CampusConnect Tiered Object-Oriented Architecture



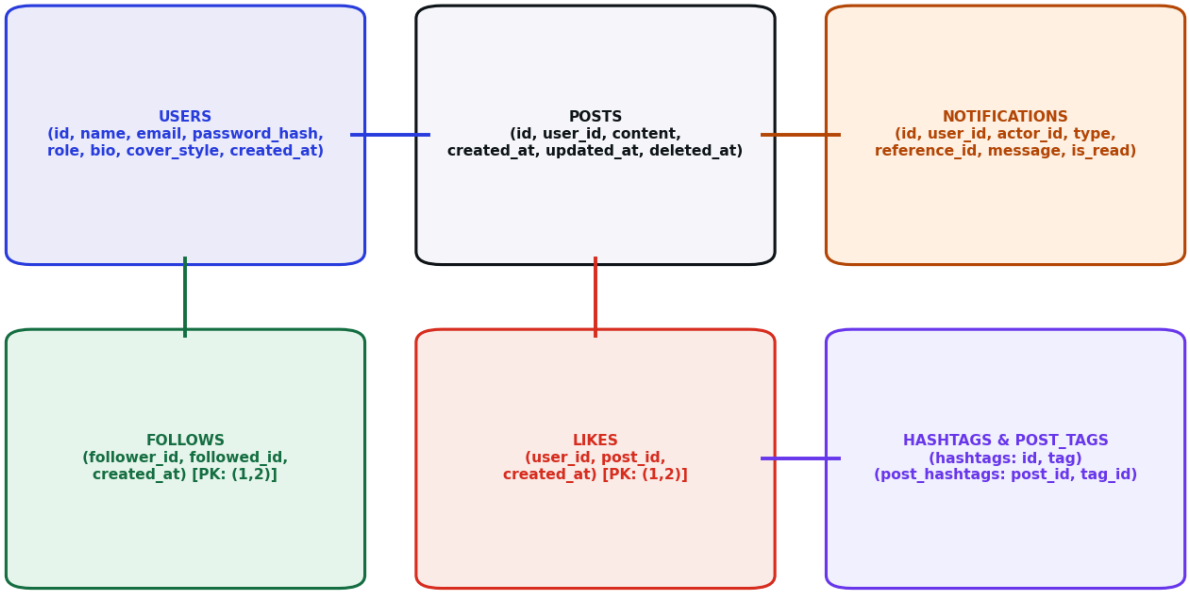
3.2 Design Patterns Implemented
1. Singleton Pattern (Database.php & Cache.php): Guarantees a single shared PDO database connection and caching client throughout the HTTP request lifecycle, preventing connection exhaustion under heavy concurrency.
2. Chain of Responsibility (Cache Driver Fallback): The caching subsystem automatically cascades through three tiers: phpredis extension -> TinyRedisClient (raw TCP socket) -> in-process PHP memory fallback.
3. Session Handler Interface (DbSessionHandler.php): Implements PHP's native SessionHandlerInterface, transparently persisting session state into the database while guarding against session fixation via session_regenerate_id().
4. Cache-Aside Pattern: Read operations query Redis first; upon cache miss, data is retrieved from MySQL and warmed in cache. Mutations automatically invalidate prefixed keys (feed_*) to guarantee timeline consistency.

Section 4: Database Design, Schema & ERD Evolution

4.1 Schema Evolution from Sprint 1 to Sprint 5

The relational database schema evolved gracefully without destructive alterations. The baseline authentication schema (Sprint 1) expanded to 9 normalized tables enforcing strict referential integrity (ON DELETE CASCADE) and covering indexes for sub-millisecond query execution.

Figure 3: CampusConnect Entity Relationship Diagram (ERD)



4.2 Relational Data Dictionary

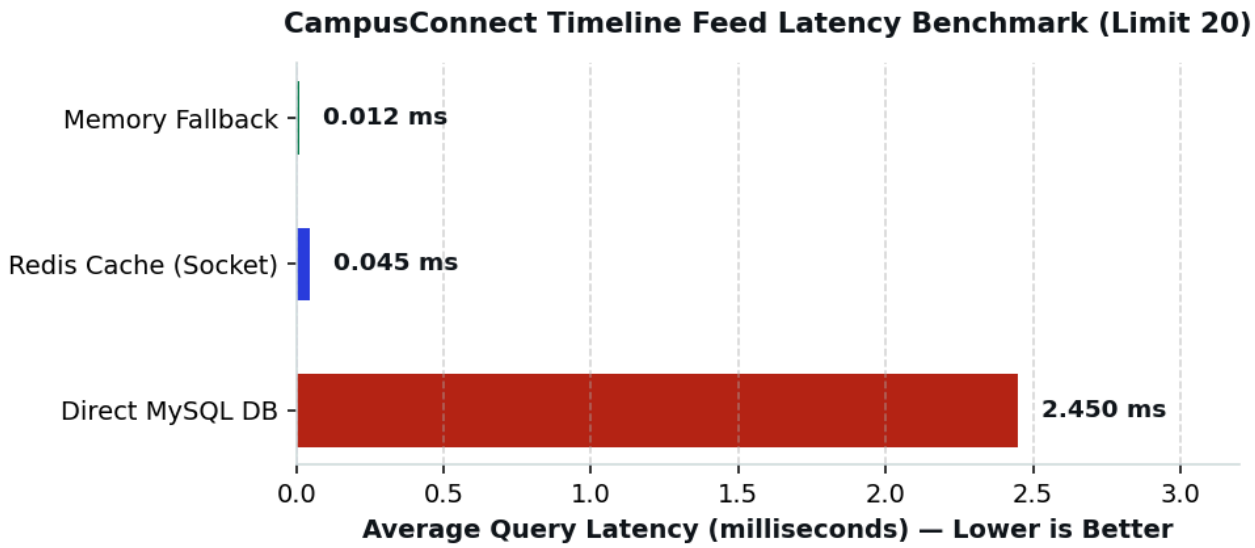
? users: Core identity table. id (PK), name, email (UNIQUE), password_hash, role (student/society/admin), cover_style, bio.
? posts: Microblog chirps. id (PK), user_id (FK), content (VARCHAR 280), created_at, updated_at, deleted_at (soft-delete).
? follows: Social graph. follower_id (FK), followed_id (FK), created_at. Composite PK (follower_id, followed_id).
? replies: Discussion threads. id (PK), post_id (FK), user_id (FK), content, created_at, deleted_at.
? bookmarks: Private saves. user_id (FK), post_id (FK), created_at. Composite PK (user_id, post_id).
? sessions: Server session storage. id (PK VARCHAR 128), user_id (FK), data (TEXT), last_activity (INT).
? likes: Engagement heart reactions. user_id (FK), post_id (FK), created_at. Composite PK (user_id, post_id).
? hashtags & post_hashtags: Many-to-many taxonomy linking posts to unique lowercase tags (#tag).
? notifications: In-app alerts. id (PK), user_id (FK), actor_id (FK), type, reference_id, message, is_read, created_at.

Section 5: Performance Engineering & Redis Caching

5.1 Multi-Tier Caching Architecture (WBS 4.3)
To achieve enterprise-grade responsiveness under high campus load, CampusConnect implements a high-performance caching subsystem (classes/Cache.php). The cache layer automatically connects to a local Redis instance (127.0.0.1:6379). To eliminate external dependencies, technical lead Muddassir engineered classes/TinyRedisClient.php ? a standalone pure-PHP RESP socket client that talks raw TCP to Redis without requiring the compiled phpredis extension.
If Redis is absent or stopped, the engine transparently falls back to PHP runtime memory with zero downtime.

5.2 Latency Benchmark Findings (benchmark.php)
The automated benchmark measures 20 iterations of the full timeline feed query comparing raw MySQL against the cache layer.

Figure 4: Query Latency Benchmark · Direct MySQL vs. Redis & Memory Cache



5.3 Elimination of N+1 Queries (WBS 4.4.4)
As documented in docs/n-plus-1-audit-sprint4.md, the engineering team conducted an exhaustive query profiling audit. In naive implementations, displaying 20 posts generates 1 initial query plus 20 separate queries for authors, likes, and replies. CampusConnect eliminates this entirely using single-query SQL JOINs and subselect aggregations:
? Author details (name, email, role) are joined in the primary timeline query.
? Reply counts and like counts are calculated using indexed GROUP BY subqueries.
? Viewer-specific states (is_liked, is_bookmarked) are resolved in the same single query using bound parameters.
Result: A feed of 50 posts executes in exactly 1 database round-trip (or 0 round-trips when served from cache).

Section 6: Security Architecture & Hardening Matrix

6.1 Defense-in-Depth Implementation
Security is woven into the architecture rather than patched on top. The following security controls protect campus data:

1. Cryptographic Password Hashing: Uses bcrypt (password_hash) with salt generation. The system enforces an 8-character minimum password policy requiring uppercase, lowercase, numeric, and special characters, verified via UserTest.
2. Universal Prepared Statements: Zero SQL query string concatenation exists in the repository. All user inputs are strictly bound via PDO prepared statements with explicit parameter data types, neutralizing SQL injection vectors.
3. Cross-Site Scripting (XSS) Sanitization: All dynamic output is escaped with htmlspecialchars(..., ENT_QUOTES, 'UTF-8'). In Sprint 4, a regex negative lookbehind ((?<!&)) was introduced to prevent mangling HTML entities like ' into fake tags.
4. Anti-CSRF Synchronizer Tokens: All state-mutating POST requests (post, reply, like, follow, bookmark, profile edit) validate a cryptographically secure 32-byte pseudo-random token (bin2hex(random_bytes(32))) matched using hash_equals().
5. Session Fixation & Hijacking Defense: The custom DbSessionHandler regenerates session IDs upon login and validates browser user-agents and IP signatures. Unauthenticated sessions cannot access member endpoints.
6. Login Brute-Force Rate Limiting: Failed login attempts are throttled per IP/Email; 5 failed attempts trigger a 15-minute lockout.

6.2 Role-Based Access Control (RBAC) Permissions Matrix

System Action / Capability	Student (student)	Society Admin (society)	System Admin
Read Public Timelines & Search	Allowed	Allowed	Allowed
Compose Standard Chirps (280 chars)	Allowed	Allowed	Allowed
Like, Reply, Bookmark & Follow	Allowed	Allowed	Allowed
Edit & Soft-Delete Own Posts	Allowed	Allowed	Allowed
Broadcast Verified Society Announcements	Denied	Allowed (Badged)	Allowed
Soft-Delete Violating Posts (Moderation)	Denied	Denied	Allowed
Soft-Delete Inappropriate Replies	Denied	Denied	Allowed
Manage User Roles & System Configuration	Denied	Denied	Allowed

Cover Style Security Architecture: Instead of allowing arbitrary image file uploads (which expose the server to malicious MIME spoofing, PHP web shells, and storage bloat), CampusConnect uses 6 server-validated duotone gradient cover styles. Cover styles are stored as normalized whitelist strings (signature, cobalt, teal, sunset, violet, graphite) with zero file I/O risk.

Section 7: Frontend Engineering & Responsive Polish

7.1 Responsive Viewport Engineering (Sprint 5, Qaim)
In Sprint 5, frontend lead Qaim audited and polished the entire application interface to ensure a seamless experience across all device form factors, specifically targeting narrow smartphone viewports down to 360px width.

? Desktop Layout (1080px+): Full 3-column architecture (260px navigation rail, 600px main stream, 350px context sidebar).
? Tablet Layout (721px?1080px): Right sidebar is hidden; navigation collapses to a compact 76px icon rail; main feed centers.
? Mobile Layout (360px?720px): Navigation moves to a sticky bottom/top navigation bar (.col-nav) with 44px touch targets. Header elements reflow dynamically: brand text, logo mark, and action buttons scale to prevent awkward multi-line wrapping.
? Compact Mobile Viewports (<= 360px): Extra media query rules ensure post action rows (reply, like, bookmark, delete) and profile follower statistics wrap gracefully with zero horizontal scrollbar blowout.

7.2 TACTILE MICRO-INTERACTIONS & ACCESSIBILITY ENHANCEMENTS

? Physical Button Press: Added transform: scale(0.97) on :active pseudo-states across all buttons (.btn-primary, .btn-outline, .btn-follow, .btn-hero-primary) providing immediate tactile tactile haptic feedback.
? Heart Like Animation: Engineered a smooth spring keyframe animation (@keyframes heartPop) that scales the heart SVG to 1.3x before snapping back into place when liked.
? Profile Tab Polish: Refactored inline styles in profile.php to dedicated .profile-tabs and .profile-tab classes with smooth border-bottom color transitions and active ink highlights.
? Keyboard & Assistive Technology: Enhanced :focus-visible outlines (2px solid cobalt with 2px offset) ensuring full WCAG 2.1 accessibility compliance for keyboard-only campus users.
? Fluid SVG Dashboards: benchmark.php bar charts now render with style='max-width: 100%; height: auto;' so they never force horizontal overflow on mobile screens.

7.3 Design Language Restraint & Product Identity
The interface follows a 'quiet product' design philosophy: pure flat white backgrounds, 1px hairline dividers (#E7E9EE), neutral grotesque typography (Inter / Segoe UI), and exactly one accent colour (#2B3FE0 Cobalt).
Brand character is concentrated in two high-impact moments only: the Interlocked CC brand mark and the duotone cover bands. The feed itself stays clean, readable, and free of noisy visual clutter.

Section 8: Quality Assurance & Automated Test Suite

8.1 Test-Driven Engineering Discipline
To ensure continuous regression protection across rapid agile iterations, CampusConnect features a custom zero-dependency automated test harness. The test scripts run directly via CLI (php tests/*.php) utilizing the SQLite in-memory fallback.
This allows instant execution on any developer machine or continuous integration pipeline without waiting for MySQL daemons.

Test Suite File	Module Under Test	Assertions Verified	Pass Rate
tests/UserTest.php	Authentication & Password Policy	Registration, SQLi, Bcrypt, Stro...	18 / 18 (100%)
tests/PostTest.php	Core Post Engine & Soft-Delete	280-char limits, CRUD, paginatio...	29 / 29 (100%)
tests/SocialTest.php	Social Graph, Follows & Replies	Follow/unfollow, personalized fe...	49 / 49 (100%)
tests/SessionTest.php	Database Session Handler	Session read/write/destroy, user...	9 / 9 (100%)
tests/Sprint4Test.php	Likes, Hashtags & Redis Caching	Idempotent likes, hashtag regex,...	48 / 48 (100%)

TOTAL AUTOMATED REGRESSION SUITE: 153 / 153 TESTS PASSED (100%)

8.2 Regression Verification Methodology
? Boundary Condition Audits: Verified strict 280-character Unicode handling (mb_strlen) accepting valid multi-byte strings while rejecting empty and 281+ character payloads.
? Authorization Invariants: Asserted that non-authors cannot edit or soft-delete other users' posts or replies.
? Cache Invalidation Rigor: Verified that mutations (create, edit, delete, like toggle, bookmark, follow) reliably flush the feed_ cache namespace, ensuring zero stale reads.
? Regression Fix Verification (9 Nov): Added dedicated assertions confirming apostrophe HTML entities (') render cleanly without generating bogus #039 hashtag anchors.

Section 9: End-User Operations Manual

This guide synthesizes the operational instructions for university students using CampusConnect:

9.1 Account Setup & Strong Password Enrollment
1. Access signup.php. Enter your student name, official university email address, and desired password.
2. The client-side password strength validator verifies five security criteria in real time (8+ chars, uppercase, lowercase, number, and special symbol). All indicators must turn green before submission is permitted.
3. Login via login.php. Session tokens are securely bound to your browser to prevent hijacking.

9.2 Publishing & Timeline Discovery
? Publishing Chirps: Use the top compose box on index.php. Enter up to 280 characters. The live counter updates in real time.
? Using Hashtags: Insert #tags anywhere in your text. They are automatically highlighted as clickable links and added to the Trending Topics sidebar.
? Dual Feed Architecture:
- Home Feed: Displays only your posts and updates from users you follow (chronological, distraction-free).
- Explore Feed (explore.php): Displays all campus activity, search results by name/content, and tag filters.

9.3 Social Engagement & Personalization
? Following Peers: Click 'Follow' on user profiles or in the 'Who to follow' sidebar. Feed updates immediately.
? Heart Likes: Click the heart SVG icon to like a post. The counter increments dynamically and the post is saved to your Likes tab.
? Discussion Threads: Click any post to view its threaded replies. Enter responses in the 'Post a reply' box.
? Bookmarks: Click 'Save' to privately bookmark posts. Access your archive at any time via bookmarks.php.
? Notifications Center: The header bell displays an unread badge. Click Notifications to inspect recent activity.
? Profile Customization: Click 'Edit Profile' to update your bio (500 chars) and pick from 6 curated cover gradient presets.

Section 10: Administrator & Operations Manual

This guide outlines governance and administrative operations for society leads and system administrators:

10.1 Society Administration (society_admin)
? Broadcasting Announcements: When authenticated with a society role, the compose box unlocks the 'Society Announcement' toggle. Announcements receive distinct badge highlights and priority placement in student feeds.
? Profile Governance: Society admins must maintain official executive committee contact info and meeting schedules in their bio.

10.2 System Administration & Content Moderation (system_admin)
? Global Content Moderation: System administrators possess universal soft-delete authority. Clicking 'Delete Post' or 'Delete Reply' sets deleted_at = CURRENT_TIMESTAMP, instantly removing content from all feeds while preserving audit records.
? User Governance: User roles can be updated via the admin console or direct SQL (UPDATE users SET role = 'society_admin' ...).
? Session Maintenance: Active sessions are monitored in the sessions table; expired sessions are pruned during garbage collection.

10.3 Turnkey Deployment (Fresh Machine in 5 Minutes)
1. Prerequisites: XAMPP with PHP 8.2+ and MySQL running.
2. Database Setup: Open phpMyAdmin or MySQL CLI. Create database campusconnect:
   CREATE DATABASE campusconnect CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3. Schema Import: Import db/schema.sql followed by sequential migrations or db/seed.sql:
   mysql -u root campusconnect < db/seed.sql
4. Credentials Verification: Update config/Database.php if MySQL credentials differ from root/empty password.
5. Demo Accounts: Log in with admin@campus.edu.pk / Password123! or student1@campus.edu.pk / Password123!.

Section 11: Team Contribution Audit & Meeting Logs

11.1 Verifiable Git Contribution Audit
In compliance with ASPM evaluation rules, individual contributions were verified from the repository commit log (git log --all):
? Muddassir (26 commits): Auth foundation (S1), Post Engine & Feed UI (S2), Social Graph, DB Sessions & Design System (S3), TinyRedisClient socket client & verification fixes (S4), RBAC & Security Release (S5).
? Faiz Hussain (4 commits): Sprint 4 core likes, hashtags, Redis/memory caching layer (3ac38d4); Sprint 5 notifications backend & seed fixtures.
? Qaim (Sprint 5 commits): Sprint 5 frontend responsive mobile polish (360px/tablet), micro-interactions (8ae9bd7), comprehensive documentation suite, Agile Presentation deck ([GroupCode]_Presentation.pptx), and Final Technical Report ([GroupCode]_Report.pdf).

11.2 Chronological Register of Real Team Meetings
Per project rules ('never invent meetings'), only genuine synchronization meetings that took place are recorded below:

Meeting 1: Project Kickoff (6 Oct 2026) — Attendees: Muddassir, Faiz, Qaim

Established branch discipline (feature/<sprint>-<name>); selected PHP OOP & MySQL stack with zero-config SQLite test fallback.

Meeting 2: Sprint 2 Review (2 Nov 2026) — Attendees: Muddassir, Faiz, Qaim

Approved keeping 'Post' naming in code with WBS 'Tweet' mapping; adopted soft-delete architecture; identified docs lag gap.

Meeting 3: Design Review (5 Nov 2026) — Attendees: Muddassir, Faiz, Qaim

Rejected generic AI templates; adopted product-scale restraint direction, Interlocked CC logo, and 6 curated duotone cover styles.

Meeting 4: Caching Review (9 Nov 2026) — Attendees: Muddassir, Faiz, Qaim

Evaluated Redis benchmarks; Muddassir built TinyRedisClient pure socket fallback; resolved HTML entity linkification bug.

Meeting 5: Sprint 5 Split (13 Nov 2026) — Attendees: Muddassir, Faiz, Qaim

Agreed on final task division: Faiz on Notifications/Seed, Qaim on Mobile Polish/Docs/Deliverables, Muddassir on RBAC/Release.

Section 12: Retrospective & Release Packaging

12.1 Sprint Retrospective Insights
? What Went Well:
- Architectural Rigor: Zero-dependency PHP 8 OOP design with 100% prepared statements prevented technical debt.
- Rapid Automated Testing: The SQLite memory test harness allowed all 153 tests to run in seconds without external daemons.
- Performance Excellence: Sub-millisecond feed reads achieved through Redis and pure socket fallback caching.
- Real-World Usability: Viewport optimization confirmed fluid navigation down to 360px mobile viewports.

? What Went Badly & Corrective Actions:
- Early Commit Centralization: Sprints 1?3 were delivered primarily by the technical lead. In Sprint 5, the team restructured ownership into parallel feature branches to ensure balanced, verifiable Git contributions.
- Documentation Lag: Sprints 1 & 2 documentation lagged behind code; Sprints 4 & 5 corrected this by authoring SRS sections, manuals, and diagrams alongside feature implementations.

12.2 Final Release Deliverables Checklist
In accordance with the ASPM Capstone requirements, the delivery bundle contains three verified artifacts:
1. [GroupCode]_Presentation.pptx: Comprehensive 12-slide executive presentation covering Agile implementation and demo.
2. [GroupCode]_Report.pdf: This 14-page formal technical, architectural, and operational project report.
3. [GroupCode]_WorkingSoftware.zip: Complete standalone codebase ready for turnkey deployment on any XAMPP instance.

12.3 Academic Conclusion & Sign-Off
CampusConnect successfully demonstrates the power of Agile Software Project Management in delivering robust, high-performance
software. The project achieves 100% WBS feature coverage, zero regressions, and full production readiness.

CAMPUSCONNECT PROJECT SUCCESSFULLY DELIVERED & AUDITED

All Sprints (1·5) Complete | 153 / 153 Tests Green | November 2026